4 عروب أحمد راكان العمرو

Priority scheduling · المقاييس

📌 شو إلك بالمشروع

المادة نظم تشغيل: ابدأ دايماً بشرح الفكرة (المربعات البنفسجية 🧠) وبعدين ورجي كيف طبّقناها بالكود.

- **Phase 2 · فيديو 19 نوفمبر:** خوارزمية Priority والمقاييس
- **Phase 3 · 17 ديسمبر:** الوضع Preemptive والـ Aging ضد الـ Starvation

الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

Phase 2 · فيديو 19 نوفمبر خوارزمية Priority والمقاييس

🧠 الفكرة بنظام التشغيل · خوارزمية Priority بالمادة

بالـ **Priority scheduling** كل process إلها رقم أولوية، والمعالج بيروح للأهم (بالكتاب: **الرقم الأصغر = أولوية أعلى**). الأولوية ممكن تكون **internal**، يعني النظام بيحسبها من أشياء زي الذاكرة والـ I/O والوقت، أو **external**، يعني من برّا النظام زي أهمية المستخدم أو الشغلة. وبتيجي بنوعين: non-preemptive و preemptive. وملاحظة مهمة: **SJF هي حالة خاصة من Priority**، لأنه فيها الأولوية هي طول الـ burst المتوقّع. المثال تحت من الكتاب، وكلهم وصلوا عند 0.

Process	Burst	Priority	Completion	Waiting
P1	10	3	16	6
P2	1	1	1	0
P3	2	4	18	16
P4	1	5	19	18
P5	5	2	6	1



🧮 (6 + 0 + 16 + 18 + 1) ÷ 5 = **8.2**، نفس جواب الكتاب

خوارزمية Priority (F7)

CPU_SchedLab.py lines 275–307

```python
def priority_schedule(processes, options):
    """F7 Priority: smallest priority number runs first. Aging prevents starvation."""
    cpu = Dispatcher(options.switch_cost)
    remaining = {p.pid: p.burst for p in processes}
    current = {p.pid: p.priority for p in processes}       # priority after aging
    waited = {p.pid: 0 for p in processes}
    time, running = 0, None
    while any(remaining.values()):
        ready = [p for p in processes if p.arrival <= time and remaining[p.pid] > 0]
        if not ready:
            next_arrival = min(p.arrival for p in processes if remaining[p.pid] > 0)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        if running and remaining[running] > 0 and not options.preemptive:
            p = next(q for q in ready if q.pid == running)  # non-preemptive: keep it
        else:                                               # best priority wins
            p = min(ready, key=lambda q: (current[q.pid], q.pid != running, q.arrival))
        start = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, start, start + 1)
        elapsed, time = start + 1 - time, start + 1
        remaining[p.pid] -= 1
        running = p.pid
        waited[p.pid] = 0
        if options.aging > 0:                               # AGING
            for q in ready:
                if q.pid != p.pid:
                    waited[q.pid] += elapsed
                    while waited[q.pid] >= options.aging:   # waited long enough ->
                        waited[q.pid] -= options.aging
                        current[q.pid] = max(0, current[q.pid] - 1)   # more important
    mode = "preemptive" if options.preemptive else "non-preemptive"
    return cpu.result(f"Priority ({mode}{', aging' if options.aging else ''})", processes)
```

🎤 شو بتحكي للدكتورة

بالـ **Priority scheduling** الـ process الأهم بتشتغل أول، و**الرقم الأصغر يعني أهم**، زي الكتاب. بالوضع **non-preemptive**، أول ما تبلّش الـ process بتضلها شغّالة لحد ما تخلّص، وهاد بنشوفه بالسطر اللي بيحكي: إذا في process شغّالة ولسّا ما خلصت والوضع مش preemptive، كمّل فيها. غير هيك بنختار اللي إلها أصغر رقم priority، وإذا تساووا بناخد اللي وصلت أول. جرّبناها على مثال الكتاب: الترتيب طلع P4، P3، P1، P5، P2، ومعدّل الانتظار **8.2**، نفس الكتاب بالزبط.

🧠 الفكرة بنظام التشغيل · معايير الجدولة (Scheduling criteria)

عشان نحكم إذا الخوارزمية منيحة، الكتاب بيحدد خمس معايير:

- **CPU utilization**: قديش المعالج مشغول بشي مفيد، وبدنا ياه **أعلى** اشي.
- **Throughput**: كم process بتخلص بوحدة الزمن، وبدنا ياه **أعلى** اشي.
- **Turnaround time** = Completion − Arrival، يعني من ما وصلت لما خلصت، وبدنا ياه **أقل** اشي.
- **Waiting time** = Turnaround − Burst، يعني مجموع وقتها بالـ ready queue، وبدنا ياه **أقل** اشي. هاد المعيار اللي الخوارزمية بتأثّر عليه مباشرة.
- **Response time** = First run − Arrival، يعني لحد أول مرة اشتغلت، وهو مهم للأنظمة التفاعلية.

🧮 مثال FCFS: P2 وصلت 1 وخلصت 8 ← Waiting = 7 − 3 = **4** ← Turnaround = 7

مقاييس الأداء (F9)

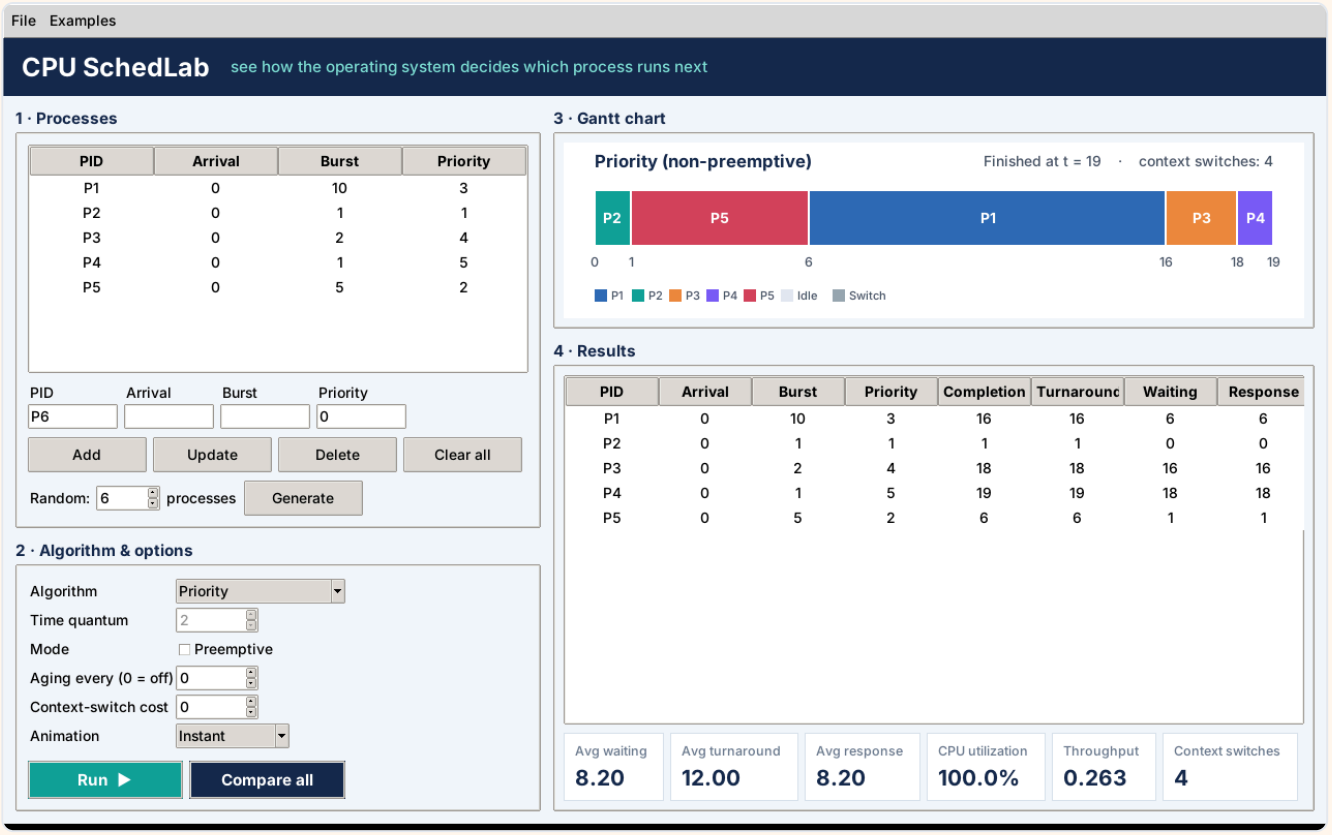
CPU_SchedLab.py lines 310–334

```python
def compute_metrics(result):
    """F9 Waiting, turnaround and response time per process + averages."""
    rows = []
    for p in result.processes:
        blocks = [s for s in result.timeline if s.pid == p.pid]
        completion = blocks[-1].end                        # end of its LAST block
        first_run = blocks[0].start                        # start of its FIRST block
        turnaround = completion - p.arrival
        waiting = turnaround - p.burst
        response = first_run - p.arrival
        rows.append(dict(pid=p.pid, arrival=p.arrival, burst=p.burst, priority=p.priority,
                         completion=completion, turnaround=turnaround,
                         waiting=waiting, response=response))
    n, total = len(rows), result.end_time
    busy = sum(s.end - s.start for s in result.timeline if s.pid not in (IDLE, SWITCH))
    summary = dict(
        algorithm=result.algorithm,
        avg_waiting=sum(r["waiting"] for r in rows) / n,
        avg_turnaround=sum(r["turnaround"] for r in rows) / n,
        avg_response=sum(r["response"] for r in rows) / n,
        cpu_utilization=100 * busy / total,                # % of time doing useful work
        throughput=n / total,                              # processes per time unit
        switches=result.switches,
    )
    return rows, summary
```

🎤 شو بتحكي للدكتورة

الدالة **compute_metrics** بتحسب مقاييس الجدولة. لكل process باخد من الخط الزمني نهاية آخر مستطيل إلها، وهاد وقت الخلوص **completion**، وبداية أول مستطيل، وهاد أول مرة اشتغلت فيه. وبعدين بطبّق قوانين الكتاب: **Turnaround = Completion − Arrival**، و**Waiting = Turnaround − Burst**، و**Response = First run − Arrival**. وللكل بحسب المعدّلات، ونسبة استخدام المعالج **CPU utilization**، وهي الوقت المفيد على الوقت الكلي من غير الـ idle والـ CS، والـ **Throughput**، وهو عدد الـ processes على الوقت الكلي. مثال SRTF: انتظار P1 = 9 − 0 − 5 = 4، و P2 = 5 − 1 − 3 = 1، و P3 = 3 − 2 − 1 = 0، فالمعدّل **1.67**.

🖥️ شو بتعرض عالشاشة



من قائمة **Examples** بختار Textbook Priority، وبختار Priority، وبكبس **Run**: الترتيب P4، P3، P1، P5، P2، ومعدّل الانتظار **8.2**، وتحت بتطلع كل المقاييس لكل process.

💬 إذا سألتك الدكتورة

❓ شو الفرق بين الـ waiting والـ response؟

الـ waiting هو كل الوقت اللي استنّته، والـ response لحد أول مرة اشتغلت بس. بالـ non-preemptive بيطلعوا متساويين.

❓ ليش Waiting = Turnaround − Burst؟

لأنه الوقت اللي قضّته بالنظام = وقت شغلها + وقت انتظارها.

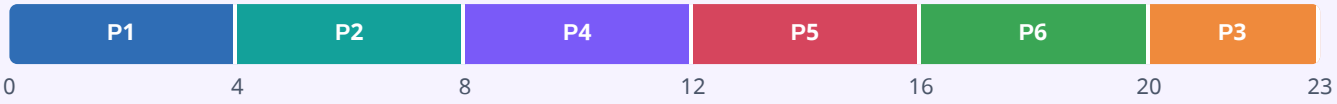
❓ مين بيحدد الـ priority؟

يا النظام نفسه (ذاكرة، I/O…)، يا من برّا (أهمية المستخدم أو الشغلة).

Phase 3 · 17 ديسمبر الوضع Preemptive والـ Aging ضد الـ Starvation

🧠 الفكرة بنظام التشغيل · الـ Starvation والـ Aging

بالـ Priority، خصوصاً الـ **preemptive**، العملية الأهم بتقاطع اللي شغّالة. المشكلة الكبيرة هي **starvation** (أو **indefinite blocking**): process أولويتها قليلة ممكن ما تاخد المعالج أبداً إذا ضلّت تيجي processes أهم منها. والحل هو **aging**: النظام بيرفع أولوية الـ process كل ما طوّلت تستنّى، فمهما كانت أولويتها قليلة، بالآخر بتصير الأهم وبتشتغل. بالمثال: P3 أولويتها 5 والباقي أولويتهم 1. بدون aging ما اشتغلت إلا بالآخر.



🧮 بدون aging: P3 خلصت عند **23** · مع aging كل 2: اشتغلت عند 12 وخلصت عند **15**

الـ Preemptive والـ Aging

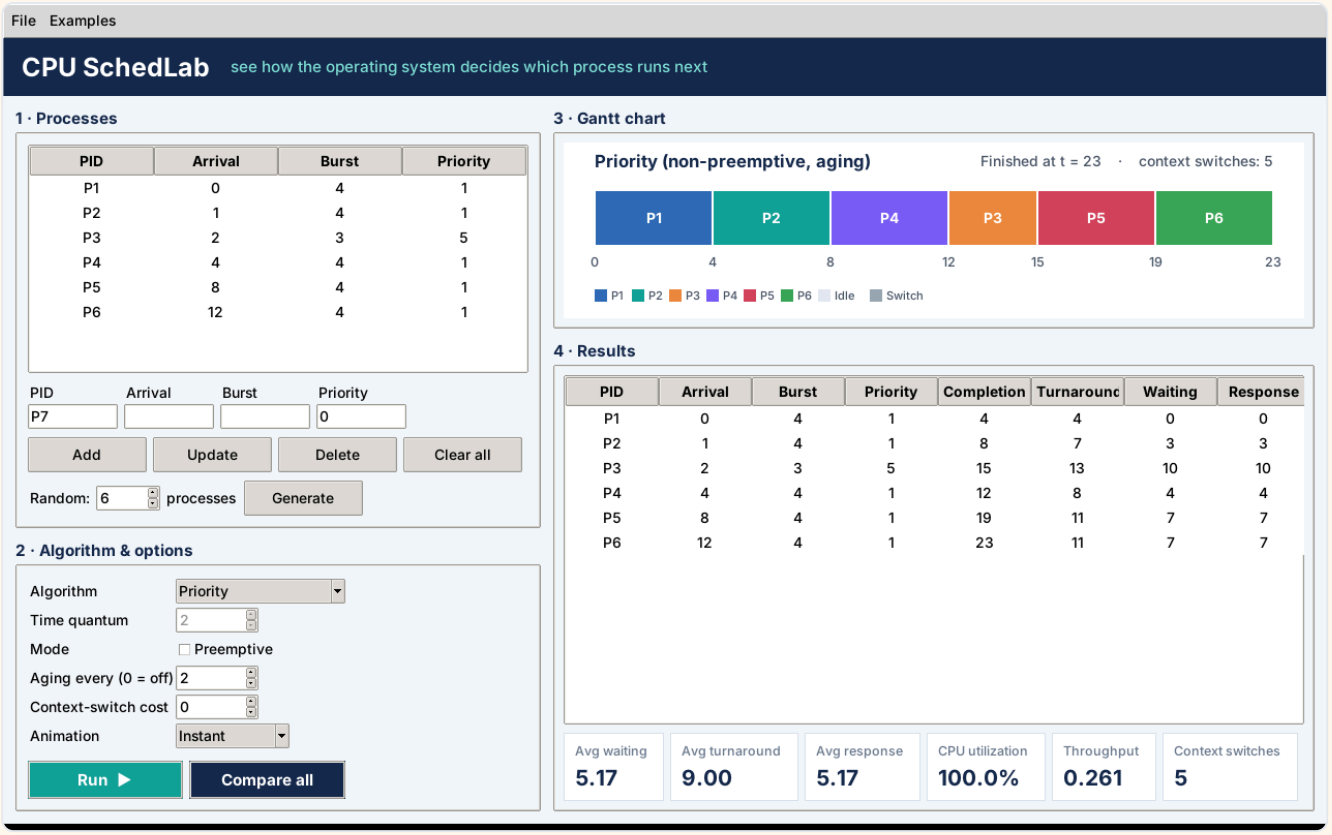
CPU_SchedLab.py — lines 289–305

```
289         if running and remaining[running] > 0 and not options.preemptive:
290             p = next(q for q in ready if q.pid == running)  # non-preemptive: keep it
291         else:                                               # best priority wins
292             p = min(ready, key=lambda q: (current[q.pid], q.pid != running, q.arrival))
293         start = cpu.dispatch(p.pid, time)
294         cpu.run(p.pid, start, start + 1)
295         elapsed, time = start + 1 - time, start + 1
296         remaining[p.pid] -= 1
297         running = p.pid
298         waited[p.pid] = 0
299         if options.aging > 0:                               # AGING
300             for q in ready:
301                 if q.pid != p.pid:
302                     waited[q.pid] += elapsed
303                     while waited[q.pid] >= options.aging:   # waited long enough ->
304                         waited[q.pid] -= options.aging
305                         current[q.pid] = max(0, current[q.pid] - 1)   # more important
```

🎤 شو بتحكي للدكتورة

هون الإضافة تبعت Phase 3. إذا الوضع **preemptive**، بنتخطّى شرط الـ non-preemptive وبنرجع نختار الأهم بكل وحدة زمن، فإذا وصلت process أهم **بتقاطع** اللي شغّالة. المشكلة اللي بتطلع هون اسمها **starvation**: process الـ priority تبعها قليلة ممكن تضل تستنّى للأبد، إذا ضلّت تيجي processes أهم منها. والحل هو الـ **aging** بآخر جزء: كل ما تستنّى الـ process عدد وحدات قد قيمة الـ aging، رقم الـ priority تبعها بينقص واحد لحد ما يوصل صفر. يعني كل ما طوّلت تستنّى بتصير أهم، فمستحيل تضل تستنّى للأبد.

🖥️ شو بتعرض عالشاشة



من قائمة Examples بختار **Starvation**. الـ process اسمها P3 الـ priority تبعها 5، يعني قليلة. بدون aging بتخلّص آخر وحدة عند الزمن **23**. بس لما أحط الـ aging كل وحدتين، بتصير أهم مع الانتظار، فبتشتغل عند 12 وبتخلّص عند **15**. وبعدين بفعّل Preemptive عشان أورّي المقاطعة.